

05. Handle Exception trong Spring Boot

5.1. Giới thiệu về Exception Handling

Trong bất kỳ ứng dụng phần mềm nào, việc xử lý ngoại lệ (exception handling) là một khía cạnh quan trọng để đảm bảo ứng dụng hoạt động ổn định, đáng tin cậy và cung cấp phản hồi thân thiện cho người dùng khi có lỗi xảy ra. Trong các ứng dụng web, đặc biệt là RESTful APIs, việc xử lý ngoại lệ đúng cách là cực kỳ quan trọng để trả về các mã trạng thái HTTP (HTTP Status Codes) và thông báo lỗi phù hợp, giúp client dễ dàng hiểu và xử lý lỗi.

Spring Boot cung cấp nhiều cơ chế mạnh mẽ và linh hoạt để xử lý ngoại lệ, từ cấp độ cục bộ (trong từng controller) đến cấp độ toàn cục (cho toàn bộ ứng dụng).

5.2. Các cơ chế xử lý ngoại lệ trong Spring Boot

5.2.1. Xử lý ngoại lệ cục bộ với `@ExceptionHandler`

Bạn có thể sử dụng annotation `@ExceptionHandler` trên một phương thức trong cùng một `@Controller` hoặc `@RestController` để xử lý các ngoại lệ cụ thể xảy ra trong controller đó. Phương thức được đánh dấu `@ExceptionHandler` sẽ được gọi khi ngoại lệ được chỉ định xảy ra.

Ví dụ:

Giả sử chúng ta có một `ProductController` và muốn xử lý ngoại lệ `ProductNotFoundException`.

Java

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import java.util.HashMap;
import java.util.Map;
```

```

@RestController
public class ProductController {

    @GetMapping("/products/{id}")
    public ResponseEntity<String> getProductById(@PathVariable Long id) {
        if (id == 99L) { // Giả lập sản phẩm không tìm thấy
            throw new ProductNotFoundException("Product with ID " + id + "
not found");
        }
        return ResponseEntity.ok("Product found: " + id);
    }

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<Map<String, String>>
handleProductNotFoundException(ProductNotFoundException ex) {
        Map<String, String> errorResponse = new HashMap<>();
        errorResponse.put("error", "Not Found");
        errorResponse.put("message", ex.getMessage());
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    // Định nghĩa ngoại lệ tùy chỉnh
    static class ProductNotFoundException extends RuntimeException {
        public ProductNotFoundException(String message) {
            super(message);
        }
    }
}

```

Ưu điểm: Đơn giản, dễ hiểu cho các trường hợp xử lý ngoại lệ cụ thể trong một controller.

Nhược điểm: Không thể tái sử dụng logic xử lý ngoại lệ cho các controller khác. Mỗi controller phải tự định nghĩa `@ExceptionHandler` cho cùng một loại ngoại lệ.

5.2.2. Xử lý ngoại lệ toàn cục với `@ControllerAdvice` /

`@RestControllerAdvice`

Để xử lý ngoại lệ một cách tập trung và toàn cục cho toàn bộ ứng dụng, bạn nên sử dụng `@ControllerAdvice` (hoặc `@RestControllerAdvice` nếu bạn chỉ xử lý RESTful APIs). Lớp được đánh dấu `@ControllerAdvice` có thể chứa các phương thức `@ExceptionHandler`, `@InitBinder`, và `@ModelAttribute` mà sẽ được áp dụng cho tất cả các controller trong ứng dụng.

Ví dụ:

Java

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<Map<String, String>>
handleProductNotFoundException(ProductNotFoundException ex) {
        Map<String, String> errorResponse = new HashMap<>();
        errorResponse.put("error", "Not Found");
        errorResponse.put("message", ex.getMessage());
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach((error) -> {
            String fieldName = ((org.springframework.validation.FieldError)
error).getField();
            String errorMessage = error.getDefaultMessage();
            errors.put(fieldName, errorMessage);
        });
        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>>
handleGeneralException(Exception ex) {
        Map<String, String> errorResponse = new HashMap<>();
        errorResponse.put("error", "Internal Server Error");
        errorResponse.put("message", "An unexpected error occurred: " +
ex.getMessage());
        return new ResponseEntity<>(errorResponse,
HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Ưu điểm:

- **Tập trung:** Tất cả logic xử lý ngoại lệ được đặt ở một nơi duy nhất.
- **Tái sử dụng:** Logic xử lý ngoại lệ có thể được tái sử dụng cho tất cả các controller.
- **Rõ ràng:** Giúp controller tập trung vào logic nghiệp vụ thay vì xử lý lỗi.

5.2.3. Sử dụng `ExceptionHandler`

Spring Boot cung cấp một `ExceptionHandler` mặc định để xử lý các lỗi HTTP. Khi một lỗi xảy ra (ví dụ: 404 Not Found, 500 Internal Server Error), Spring Boot sẽ chuyển tiếp request đến `/error` endpoint, được xử lý bởi `HandlerExceptionResolver` mặc định. Bạn có thể tùy chỉnh trang lỗi này hoặc tạo `ExceptionHandler` của riêng mình.

Tùy chỉnh trang lỗi HTML:

Bạn có thể tạo các trang lỗi HTML tĩnh trong thư mục `src/main/resources/static/error/` với tên file là mã trạng thái HTTP (ví dụ: `404.html` , `500.html`).

Tùy chỉnh `ExceptionHandler` :

Bạn có thể tạo một lớp implement `ExceptionHandler` để xử lý các lỗi một cách tùy chỉnh. Tuy nhiên, với `@ExceptionHandler` , việc này thường không cần thiết cho REST APIs.

5.2.4. `ResponseStatusException`

`ResponseStatusException` là một cách tiện lợi để ném ra các ngoại lệ với mã trạng thái HTTP cụ thể. Nó cho phép bạn ném một ngoại lệ trực tiếp từ bất kỳ lớp nào (service, repository) và Spring sẽ tự động ánh xạ nó thành một phản hồi HTTP với mã trạng thái và thông báo lỗi tương ứng.

Ví dụ:

Java

```
import org.springframework.http.HttpStatus;  
import org.springframework.web.server.ResponseStatusException;  
import org.springframework.stereotype.Service;
```

```

@Service
public class ProductService {

    public String getProductDetails(Long id) {
        if (id == 99L) {
            throw new RuntimeException(HttpStatus.NOT_FOUND, "Product
not found with ID: " + id);
        }
        return "Details for product: " + id;
    }
}

```

Khi `RuntimeException` được ném ra, Spring sẽ tự động trả về phản hồi HTTP 404 Not Found với thông báo lỗi đã cung cấp.

5.3. Xây dựng cấu trúc phản hồi lỗi chuẩn

Để client dễ dàng xử lý lỗi, bạn nên trả về một cấu trúc phản hồi lỗi chuẩn (ví dụ: JSON) cho tất cả các loại lỗi. Điều này có thể được thực hiện thông qua `@ControllerAdvice`.

Ví dụ về lớp `ErrorResponse` :

Java

```

import java.time.LocalDateTime;
import java.util.List;

public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private List<String> details;

    public ErrorResponse(int status, String error, String message, String
path) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
        this.path = path;
    }
}

```

```

    public ErrorResponse(int status, String error, String message, String
path, List<String> details) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
        this.path = path;
        this.details = details;
    }

    // Getters for all fields
    public LocalDateTime getTimestamp() { return timestamp; }
    public int getStatus() { return status; }
    public String getError() { return error; }
    public String getMessage() { return message; }
    public String getPath() { return path; }
    public List<String> getDetails() { return details; }
}

```

Cập nhật `GlobalExceptionHandler` để sử dụng `ErrorResponse` :

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.util.List;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleProductNotFoundException(ProductNotFoundException ex, WebRequest
request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            HttpStatus.NOT_FOUND.getReasonPhrase(),
            ex.getMessage(),
            request.getDescription(false)
        );
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }
}

```

```

    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
    handleValidationExceptions(MethodArgumentNotValidException ex, WebRequest
    request) {
        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
            .map(error -> error.getField() + ": " +
    error.getDefaultMessage())
            .collect(Collectors.toList());

        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Validation failed",
            request.getDescription(false),
            errors
        );
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneralException(Exception ex,
    WebRequest request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            HttpStatus.INTERNAL_SERVER_ERROR.getReasonPhrase(),
            "An unexpected error occurred: " + ex.getMessage(),
            request.getDescription(false)
        );
        return new ResponseEntity<>(errorResponse,
    HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

5.4. Bản chất của Exception Handling trong Spring Boot

Bản chất của Exception Handling trong Spring Boot là cung cấp một cách tiếp cận tập trung và có cấu trúc để quản lý các lỗi xảy ra trong ứng dụng. Thay vì để các ngoại lệ không được xử lý làm sập ứng dụng hoặc trả về các phản hồi không rõ ràng, Spring Boot cho phép bạn định nghĩa các chiến lược xử lý lỗi linh hoạt, từ việc ánh xạ ngoại lệ thành mã trạng thái HTTP phù hợp đến việc trả về các thông báo lỗi có ý nghĩa cho client. Cơ chế

`@ControllerAdvice` là cốt lõi, cho phép tách biệt logic xử lý lỗi khỏi logic nghiệp vụ của

controller, giúp mã nguồn sạch hơn và dễ bảo trì hơn. Điều này không chỉ cải thiện trải nghiệm người dùng mà còn giúp các ứng dụng Spring Boot trở nên mạnh mẽ và đáng tin cậy hơn trong môi trường sản xuất.

5.5. Tạo và sử dụng Custom Exceptions

Trong nhiều trường hợp, việc sử dụng các ngoại lệ tiêu chuẩn của Java hoặc Spring là không đủ để thể hiện các lỗi nghiệp vụ cụ thể của ứng dụng. Việc tạo các ngoại lệ tùy chỉnh (Custom Exceptions) giúp mã nguồn rõ ràng hơn, dễ đọc hơn và cho phép bạn xử lý các lỗi nghiệp vụ một cách có cấu trúc.

5.5.1. Khi nào nên tạo Custom Exception?

Bạn nên tạo Custom Exception khi:

- **Lỗi nghiệp vụ cụ thể:** Một lỗi xảy ra do vi phạm quy tắc nghiệp vụ của ứng dụng (ví dụ: `InsufficientStockException` , `UserAlreadyExistsException`).
- **Cần truyền tải thông tin chi tiết:** Ngoại lệ cần mang theo các thông tin cụ thể để client hoặc các tầng khác có thể xử lý (ví dụ: mã lỗi, danh sách các trường bị lỗi).
- **Cần phân biệt rõ ràng:** Để phân biệt lỗi nghiệp vụ với các lỗi hệ thống hoặc lỗi kỹ thuật.

5.5.2. Cách tạo Custom Exception

Custom Exception thường kế thừa từ `RuntimeException` (để trở thành unchecked exception) hoặc `Exception` (để trở thành checked exception). Trong môi trường Spring Boot RESTful API, `RuntimeException` thường được ưu tiên để tránh việc phải khai báo `throws` ở khắp mọi nơi.

****Ví dụ: **** `ResourceNotFoundException`

Java

```
import org.springframework.http.HttpStatus;  
import org.springframework.web.bind.annotation.ResponseStatus;  
  
// Tùy chọn: Sử dụng @ResponseStatus để tự động ánh xạ sang HTTP Status Code  
@ResponseStatus(HttpStatus.NOT_FOUND)
```



```
public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }

    public ResourceNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }

}
```

Ví dụ: `InvalidInputException` với thông tin chi tiết

Java

```
import java.util.List;
import java.util.ArrayList;

public class InvalidInputException extends RuntimeException {
    private final List<String> errors;

    public InvalidInputException(String message, List<String> errors) {
        super(message);
        this.errors = errors;
    }

    public InvalidInputException(String message) {
        super(message);
        this.errors = new ArrayList<>();
    }

    public List<String> getErrors() {
        return errors;
    }

}
```

5.5.3. Sử dụng Custom Exception trong ứng dụng

Bạn có thể ném các Custom Exception từ bất kỳ tầng nào của ứng dụng (Service, Repository, Controller).

Ví dụ trong Service Layer:

Java

```

import org.springframework.stereotype.Service;

@Service
public class UserService {

    public User getUserById(Long id) {
        // Giả sử logic tìm kiếm người dùng
        if (id == 100L) {
            throw new ResourceNotFoundException("User with ID " + id + " not found");
        }
        return new User(id, "John Doe");
    }

    public void createUser(User user) {
        if (user.getEmail().equals("existing@example.com")) {
            throw new InvalidInputException("Email already exists",
List.of("email: Email already registered"));
        }
        // Logic tạo người dùng
    }
}

```

5.5.4. Xử lý Custom Exception trong @ControllerAdvice

Để xử lý các Custom Exception này một cách tập trung, bạn thêm các phương thức

@ExceptionHandler tương ứng vào lớp @RestControllerAdvice .

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.time.LocalDateTime;
import java.util.List;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // ... (các handler đã có)

    @ExceptionHandler(ResourceNotFoundException.class)

```

```

    public ResponseEntity<ErrorResponse>
    handleResourceNotFoundException(ResourceNotFoundException ex, WebRequest
    request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            HttpStatus.NOT_FOUND.getReasonPhrase(),
            ex.getMessage(),
            request.getDescription(false)
        );
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse>
    handleInvalidInputException(InvalidInputException ex, WebRequest request) {
        ErrorResponse errorResponse = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            ex.getMessage(),
            request.getDescription(false),
            ex.getErrors()
        );
        return new ResponseEntity<>(errorResponse, HttpStatus.BAD_REQUEST);
    }
}

```

5.6. Error Handling trong REST APIs và Problem Details

Khi xây dựng RESTful APIs, việc trả về các phản hồi lỗi nhất quán và có ý nghĩa là rất quan trọng để client có thể dễ dàng hiểu và xử lý. Ngoài việc sử dụng các mã trạng thái HTTP chuẩn, việc cung cấp thêm thông tin chi tiết về lỗi theo một định dạng chuẩn cũng rất hữu ích. RFC 7807 (Problem Details for HTTP APIs) định nghĩa một định dạng chuẩn cho các phản hồi lỗi HTTP.

5.6.1. Cấu trúc Problem Details (RFC 7807)

Một đối tượng Problem Details thường bao gồm các trường sau:

- `type` (URI): Một URI định danh loại lỗi. Có thể là một URL dẫn đến tài liệu mô tả lỗi.
- `title` (string): Một tiêu đề ngắn gọn, dễ đọc, tóm tắt loại lỗi.

- `status` (number): Mã trạng thái HTTP của phản hồi (ví dụ: 400, 404, 500).
- `detail` (string): Một mô tả chi tiết, cụ thể về lỗi xảy ra.
- `instance` (URI): Một URI định danh cụ thể của sự cố xảy ra.

Bạn cũng có thể thêm các trường mở rộng tùy chỉnh vào đối tượng Problem Details.

5.6.2. Triển khai Problem Details trong Spring Boot

Spring Boot 2.3 trở lên đã hỗ trợ tốt hơn cho Problem Details thông qua thư viện `spring-web` và `spring-webmvc`. Khi bạn sử dụng `@RestControllerAdvice` và `ResponseBodyExceptionHandler`, Spring sẽ tự động tạo ra các phản hồi lỗi tuân thủ RFC 7807 cho một số ngoại lệ Spring MVC tiêu chuẩn (ví dụ: `MethodArgumentNotValidException`).

Để tùy chỉnh hoặc tạo Problem Details cho các ngoại lệ của riêng bạn, bạn có thể kế thừa từ `ResponseBodyExceptionHandler` và ghi đè các phương thức `handleExceptionInternal` hoặc các phương thức `handle...` cụ thể.

Ví dụ: Tùy chỉnh `GlobalExceptionHandler` để trả về Problem Details

Đầu tiên, chúng ta có thể định nghĩa một lớp `ProblemDetailResponse` (hoặc sử dụng `org.springframework.http.ProblemDetail` trực tiếp từ Spring 6 trở lên).

Java

```
import java.net.URI;
import java.time.Instant;
import java.util.Map;

// Lớp này có thể được sử dụng để tạo phản hồi Problem Details
// Hoặc sử dụng org.springframework.http.ProblemDetail nếu bạn dùng Spring 6+
public class ProblemDetailResponse {
    private URI type;
    private String title;
    private int status;
    private String detail;
    private URI instance;
    private Instant timestamp;
    private Map<String, Object> properties; // Để thêm các trường tùy chỉnh

    public ProblemDetailResponse(URI type, String title, int status, String
```

```

detail, URI instance) {
    this.type = type;
    this.title = title;
    this.status = status;
    this.detail = detail;
    this.instance = instance;
    this.timestamp = Instant.now();
}

// Getters and Setters
public URI getType() { return type; }
public void setType(URI type) { this.type = type; }
public String getTitle() { return title; }
public void setTitle(String title) { this.title = title; }
public int getStatus() { return status; }
public void setStatus(int status) { this.status = status; }
public String getDetail() { return detail; }
public void setDetail(String detail) { this.detail = detail; }
public URI getInstance() { return instance; }
public void setInstance(URI instance) { this.instance = instance; }
public Instant getTimestamp() { return timestamp; }
public void setTimestamp(Instant timestamp) { this.timestamp = timestamp; }
}

public Map<String, Object> getProperties() { return properties; }
public void setProperties(Map<String, Object> properties) {
    this.properties = properties; }
}

```

Cập nhật GlobalExceptionHandler :

Java

```

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.net.URI;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

```

```

@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    // Xử lý ngoại lệ tùy chỉnh ResourceNotFoundException
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Object>
handleResourceNotFoundException(ResourceNotFoundException ex, WebRequest
request) {
        ProblemDetailResponse problemDetail = new ProblemDetailResponse(
            URI.create("/errors/not-found"),
            "Resource Not Found",
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            URI.create(request.getDescription(false).substring(4)) //
Loại bỏ

```

5.7. Thứ tự xử lý ngoại lệ (Ordering)

Khi có nhiều cơ chế xử lý ngoại lệ được cấu hình trong ứng dụng Spring Boot, Spring sẽ tuân theo một thứ tự ưu tiên nhất định để quyết định handler nào sẽ được sử dụng:

1. ****`@ExceptionHandler` trong Controller cụ thể:**** Các phương thức `@ExceptionHandler` được định nghĩa trực tiếp trong một `@Controller` hoặc `@RestController` sẽ có ưu tiên cao nhất cho các ngoại lệ xảy ra trong chính controller đó.
2. ****`@ControllerAdvice` / `@RestControllerAdvice`:**** Các phương thức `@ExceptionHandler` trong các lớp được đánh dấu `@ControllerAdvice` hoặc `@RestControllerAdvice` sẽ được áp dụng cho tất cả các controller trong ứng dụng. Nếu có nhiều `@ControllerAdvice` xử lý cùng một loại ngoại lệ, Spring sẽ chọn một trong số chúng (thứ tự có thể không xác định hoặc dựa trên `@Order` nếu được cấu hình).
3. ****`HandlerExceptionResolver`:**** Spring cung cấp một chuỗi các `HandlerExceptionResolver` mặc định để xử lý các ngoại lệ. Ví dụ, `DefaultHandlerExceptionResolver` xử lý các ngoại lệ Spring MVC tiêu chuẩn và ánh xạ chúng thành các mã trạng thái HTTP. `ResponseEntityExceptionHandler` (mà `GlobalExceptionHandler` của chúng ta có thể kế thừa) cũng là một dạng của `HandlerExceptionResolver`.
4. ****`ErrorController`:**** Nếu không có handler nào khác xử lý ngoại lệ, Spring Boot sẽ chuyển tiếp request đến `/error` endpoint, được xử lý bởi `BasicErrorController` mặc định hoặc `ErrorController` tùy chỉnh của bạn. Đây là cơ chế cuối cùng để cung cấp một phản hồi lỗi chung.

****Nguyên tắc:**** Spring luôn cố gắng tìm handler cụ thể nhất cho một ngoại lệ. Nếu một ngoại lệ được xử lý ở cấp độ controller, nó sẽ không được chuyển đến `@ControllerAdvice`. Nếu `@ControllerAdvice` xử lý nó, nó sẽ không đến

`ExceptionHandler`.

5.8. Các phương pháp hay nhất (Best Practices) cho Exception Handling

Để xây dựng một hệ thống xử lý ngoại lệ mạnh mẽ và dễ bảo trì trong Spring Boot, hãy tuân thủ các nguyên tắc sau:

1. **Sử dụng `@RestControllerAdvice`** cho xử lý lỗi toàn cục: Đây là cách tốt nhất để tập trung tất cả logic xử lý ngoại lệ của bạn vào một nơi duy nhất, giúp mã nguồn sạch sẽ và dễ quản lý hơn.
2. **Tạo Custom Exceptions cho lỗi nghiệp vụ:** Thay vì ném các `RuntimeException` chung chung, hãy định nghĩa các ngoại lệ tùy chỉnh để thể hiện rõ ràng các điều kiện lỗi nghiệp vụ. Điều này giúp mã nguồn dễ đọc hơn và cho phép client xử lý lỗi một cách cụ thể hơn.
3. **Trả về HTTP Status Codes phù hợp:** Luôn ánh xạ các ngoại lệ của bạn sang các mã trạng thái HTTP chuẩn và có ý nghĩa (ví dụ: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error). Điều này giúp client hiểu rõ loại lỗi và cách xử lý.
4. **Cung cấp thông báo lỗi rõ ràng và hữu ích:** Thông báo lỗi nên đủ chi tiết để client hiểu vấn đề là gì, nhưng không quá chi tiết để lộ thông tin nhạy cảm của hệ thống. Sử dụng Problem Details (RFC 7807) để cung cấp cấu trúc phản hồi lỗi chuẩn.
5. **Phân biệt lỗi nghiệp vụ và lỗi hệ thống:** Lỗi nghiệp vụ (ví dụ: `InvalidInputException`, `ResourceNotFoundException`) nên được xử lý một cách duyên dáng và trả về các mã trạng thái client error (4xx). Lỗi hệ thống (ví dụ: `NullPointerException`, lỗi kết nối database) thường chỉ ra một vấn đề nghiêm trọng và nên trả về 500 Internal Server Error.
6. **Ghi log ngoại lệ đúng cách:** Luôn ghi log các ngoại lệ ở cấp độ phù hợp (ví dụ: `ERROR` cho các lỗi không mong muốn, `WARN` cho các vấn đề có thể phục hồi). Đảm bảo log chứa đủ thông tin để gỡ lỗi (stack trace, context).
7. **Tránh nuốt ngoại lệ:** Không bao giờ bắt một ngoại lệ và không làm gì với nó. Nếu bạn bắt một ngoại lệ, hãy xử lý nó, ghi log nó, hoặc ném lại một ngoại lệ khác có ý nghĩa hơn.
8. **Sử dụng `ResponseStatusException`** cho các trường hợp đơn giản: Đối với các trường hợp ngoại lệ đơn giản mà không cần logic xử lý phức tạp, `ResponseStatusException` là một cách nhanh chóng và tiện lợi để ném một ngoại lệ với mã trạng thái HTTP cụ thể.
9. **Kiểm thử xử lý ngoại lệ:** Viết các bài kiểm thử để đảm bảo rằng các cơ chế xử lý ngoại lệ của bạn hoạt động đúng như mong đợi và trả về các phản hồi lỗi chính xác.

Bằng cách tuân thủ các phương pháp này, bạn có thể xây dựng một hệ thống xử lý ngoại lệ mạnh mẽ, giúp ứng dụng của bạn ổn định hơn, dễ gỡ lỗi hơn và cung cấp trải nghiệm tốt hơn cho người dùng và các client API.

5.1.1. Cơ chế xử lý ngoại lệ của Spring Framework

Spring Framework cung cấp một cơ chế xử lý ngoại lệ mạnh mẽ dựa trên mô hình ****HandlerExceptionResolver****. Đây là một **interface** trong Spring MVC cho phép bạn tùy chỉnh cách các ngoại lệ được xử lý trong ứng dụng web. Khi một ngoại lệ xảy ra trong quá trình xử lý request, Spring MVC DispatcherServlet sẽ duyệt qua danh sách các `HandlerExceptionResolver` đã đăng ký để tìm một resolver có thể xử lý ngoại lệ đó.

Các `HandlerExceptionResolver` phổ biến được Spring cung cấp bao gồm:

- * **`DefaultHandlerExceptionResolver`**: Xử lý các ngoại lệ tiêu chuẩn của Spring MVC và ánh xạ chúng thành các mã trạng thái HTTP tương ứng (ví dụ: `HttpRequestMethodNotSupportedException` -> 405 Method Not Allowed, `HttpMediaTypeNotSupportedException` -> 415 Unsupported Media Type).
- * **`ExceptionHandlerExceptionResolver`**: Đây là resolver quan trọng nhất, chịu trách nhiệm phát hiện và gọi các phương thức được đánh dấu `@ExceptionHandler` trong các `@Controller` và `@ControllerAdvice`.
- * **`ResponseStatusExceptionHandlerResolver`**: Xử lý các ngoại lệ được đánh dấu `@ResponseStatus` và `ResponseStatusException`, ánh xạ chúng thành các mã trạng thái HTTP tương ứng.

Khi một ngoại lệ được ném ra, luồng xử lý sẽ như sau:

1. Ngoại lệ được ném ra từ Controller, Service, hoặc Repository.
2. Spring MVC `DispatcherServlet` bắt ngoại lệ.
3. `DispatcherServlet` duyệt qua danh sách các `HandlerExceptionResolver` đã đăng ký.
4. `ExceptionHandlerExceptionResolver` sẽ kiểm tra xem có phương thức `@ExceptionHandler` nào phù hợp trong Controller hiện tại hoặc trong bất kỳ `@ControllerAdvice` nào không.
5. Nếu tìm thấy, phương thức `@ExceptionHandler` phù hợp sẽ được gọi để xử lý ngoại lệ và tạo ra một `ResponseEntity` hoặc `ModelAndView`.
6. Nếu không có `@ExceptionHandler` phù hợp, các `HandlerExceptionResolver` khác (như `ResponseStatusExceptionHandlerResolver` hoặc `DefaultHandlerExceptionResolver`) sẽ cố gắng xử lý.
7. Nếu không có `HandlerExceptionResolver` nào xử lý được, ngoại lệ sẽ được chuyển đến Servlet Container, và cuối cùng Spring Boot sẽ chuyển hướng đến `/error` endpoint (được xử lý bởi `BasicErrorController`).

Cơ chế này cho phép bạn có sự kiểm soát chi tiết về cách các lỗi được trình bày cho người dùng, đồng thời giữ cho logic nghiệp vụ của bạn sạch sẽ và không bị lẫn lộn với logic xử lý lỗi.

5.6. Thứ tự xử lý ngoại lệ (Ordering) và Best Practices

Khi bạn có nhiều cơ chế xử lý ngoại lệ trong ứng dụng Spring Boot, việc hiểu rõ thứ tự ưu tiên và các phương pháp hay nhất là rất quan trọng để đảm bảo lỗi được xử lý đúng cách.

5.6.1. Thứ tự ưu tiên của các cơ chế xử lý ngoại lệ

Spring Framework có một thứ tự ưu tiên cụ thể khi tìm kiếm một handler cho một ngoại lệ:

1. **`@ExceptionHandler`** trong Controller cục bộ: Nếu một phương thức `@ExceptionHandler` được định nghĩa trong cùng một Controller nơi ngoại lệ xảy ra, nó sẽ được ưu tiên xử lý trước.
2. **`@ControllerAdvice`** / **`@RestControllerAdvice`**: Nếu không có `@ExceptionHandler` cục bộ, Spring sẽ tìm kiếm các lớp được đánh dấu `@ControllerAdvice` hoặc `@RestControllerAdvice` để tìm một phương thức `@ExceptionHandler` phù hợp.
 - * Nếu có nhiều `@ControllerAdvice` có thể xử lý cùng một ngoại lệ, Spring sẽ chọn một trong số đó. Bạn có thể sử dụng `@Order` hoặc `Ordered` interface để định nghĩa thứ tự ưu tiên cho các `@ControllerAdvice` nếu cần.
3. **`HandlerExceptionResolver`**'s mặc định của Spring: Nếu không có `@ExceptionHandler` nào được tìm thấy, Spring sẽ sử dụng các `HandlerExceptionResolver` mặc định của nó (ví dụ: `DefaultHandlerExceptionResolver`, `ExceptionHandlerExceptionResolver`). Các resolver này xử lý các ngoại lệ Spring MVC phổ biến (như `HttpRequestMethodNotSupportedException`, `MissingServletRequestParameterException`) và ánh xạ chúng thành các mã trạng thái HTTP phù hợp.
4. **`ErrorController`** của Spring Boot: Nếu không có cơ chế nào ở trên xử lý được ngoại lệ, Spring Boot sẽ chuyển tiếp yêu cầu đến `/error` endpoint, được xử lý bởi `BasicErrorController` mặc định. Controller này sẽ trả về một trang lỗi HTML hoặc một phản hồi JSON mặc định.

Nguyên tắc chung: Xử lý ngoại lệ càng cụ thể càng tốt ở cấp độ gần nhất với nơi ngoại lệ xảy ra. Nếu một ngoại lệ có thể được xử lý chung cho nhiều controller, hãy sử dụng `@ControllerAdvice`.

5.6.2. Best Practices cho Exception Handling

1. **Sử dụng `@ControllerAdvice` cho xử lý lỗi toàn cục:** Đây là cách tiếp cận được khuyến nghị nhất cho các ứng dụng Spring Boot, đặc biệt là RESTful APIs. Nó giúp tập trung logic xử lý lỗi, giữ cho các controller sạch sẽ và dễ bảo trì.
2. **Tạo Custom Exceptions cho lỗi nghiệp vụ:** Thay vì ném các `RuntimeException` chung chung, hãy tạo các ngoại lệ tùy chỉnh để biểu thị các lỗi nghiệp vụ cụ thể (ví dụ: `UserNotFoundException`, `InvalidOrderException`). Điều này giúp mã nguồn rõ ràng hơn và cho phép bạn xử lý các loại lỗi khác nhau một cách có cấu trúc.

3. ****Ánh xạ ngoại lệ thành mã trạng thái HTTP phù hợp:**** Đảm bảo rằng các ngoại lệ được ánh xạ thành các mã trạng thái HTTP có ý nghĩa (ví dụ: 400 Bad Request cho lỗi validation, 404 Not Found cho tài nguyên không tồn tại, 401 Unauthorized, 403 Forbidden, 500 Internal Server Error cho lỗi hệ thống). Sử dụng `@ResponseStatus` trên Custom Exception hoặc trả về `ResponseEntity` với `HttpStatus` phù hợp trong `@ExceptionHandler`.

4. ****Trả về cấu trúc phản hồi lỗi chuẩn:**** Luôn trả về một cấu trúc JSON (hoặc XML) nhất quán cho tất cả các phản hồi lỗi. Cấu trúc này nên bao gồm các thông tin như mã trạng thái, thông báo lỗi, timestamp, và có thể là chi tiết lỗi (ví dụ: danh sách các lỗi validation).

5. ****Ghi log ngoại lệ một cách thích hợp:**** Ghi log các ngoại lệ ở cấp độ phù hợp (ví dụ: `ERROR` cho các lỗi nghiêm trọng, `WARN` cho các vấn đề ít nghiêm trọng hơn). Tránh ghi log quá nhiều thông tin nhạy cảm. Sử dụng các framework logging như SLF4J/Logback.

6. ****Tránh bắt lỗi chung chung (`Exception.class`) quá sớm:**** Tránh sử dụng `try-catch (Exception e)` ở tầng thấp (ví dụ: Repository) trừ khi bạn thực sự có thể xử lý hoặc phục hồi từ lỗi đó. Hãy để các ngoại lệ được ném lên các tầng cao hơn (Service, Controller) để được xử lý tập trung bởi `@ControllerAdvice`.

7. ****Sử dụng `Problem Details for HTTP APIs` (RFC 7807):**** Đối với các RESTful API hiện đại, việc tuân thủ RFC 7807 để định dạng phản hồi lỗi là một phương pháp hay. Spring Boot 2.3+ hỗ trợ `ProblemDetail` để tạo các phản hồi lỗi tuân thủ chuẩn này.

****Ví dụ về `ProblemDetail`:**

```
```java
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;
import java.time.Instant;

@RestControllerAdvice
public class GlobalProblemDetailExceptionHandler {

 @ExceptionHandler(ProductNotFoundException.class)
 public ProblemDetail
handleProductNotFoundException(ProductNotFoundException ex) {
 ProblemDetail problemDetail =
ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
 problemDetail.setTitle("Product Not Found");
 }
}
```

```

problemDetail.setType(URI.create("https://example.com/problems/product-not-found"));
 problemDetail.setProperty("timestamp", Instant.now());
 return problemDetail;
 }

 @ExceptionHandler(InvalidInputException.class)
 public ProblemDetail
handleInvalidInputException(InvalidInputException ex) {
 ProblemDetail problemDetail =
ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
 problemDetail.setTitle("Invalid Input");

problemDetail.setType(URI.create("https://example.com/problems/invalid-input"));
 problemDetail.setProperty("errors", ex.getErrors()); // Thêm chi
tiết lỗi validation
 problemDetail.setProperty("timestamp", Instant.now());
 return problemDetail;
 }
}
...

```

Khi sử dụng `ProblemDetail`, Spring sẽ tự động thiết lập `Content-Type` là `application/problem+json`.

8. **Kiểm tra và xử lý `ConstraintViolationException`:** Khi sử dụng Bean Validation ở tầng Service hoặc DAO (không phải qua `@Valid` trong Controller), các lỗi validation có thể được ném ra dưới dạng `ConstraintViolationException`. Bạn cần có một `@ExceptionHandler` riêng để xử lý loại ngoại lệ này và ánh xạ nó thành phản hồi lỗi phù hợp.

Bằng cách tuân thủ các nguyên tắc và phương pháp hay nhất này, bạn có thể xây dựng một hệ thống xử lý ngoại lệ mạnh mẽ, đáng tin cậy và thân thiện với người dùng trong các ứng dụng Spring Boot của mình.

## ## 5.6. Thứ tự xử lý ngoại lệ (Ordering of Exception Handlers)

Khi có nhiều cơ chế xử lý ngoại lệ được cấu hình trong ứng dụng Spring Boot của bạn, điều quan trọng là phải hiểu thứ tự ưu tiên mà Spring sử dụng để tìm và áp dụng handler phù hợp. Thứ tự này đảm bảo rằng ngoại lệ được xử lý bởi handler cụ thể nhất và phù hợp nhất.

Thứ tự ưu tiên từ cao đến thấp (từ cụ thể đến tổng quát):

1. **`@ExceptionHandler`** trong cùng Controller:
  - \* Các phương thức được đánh dấu **`@ExceptionHandler`** trong chính Controller nơi ngoại lệ xảy ra sẽ có ưu tiên cao nhất. Chúng chỉ áp dụng cho các ngoại lệ phát sinh trong Controller đó.
  - \* Đây là cơ chế xử lý ngoại lệ cục bộ nhất.
2. **`@ExceptionHandler`** trong **`@ControllerAdvice`** / **`@RestControllerAdvice`**:
  - \* Các phương thức **`@ExceptionHandler`** trong các lớp được đánh dấu **`@ControllerAdvice`** hoặc **`@RestControllerAdvice`** sẽ xử lý các ngoại lệ trên phạm vi toàn cục cho tất cả các Controller mà advice đó áp dụng.
  - \* Nếu có nhiều **`@ControllerAdvice`** được định nghĩa, Spring sẽ cố gắng tìm handler phù hợp nhất. Bạn có thể kiểm soát thứ tự ưu tiên của các **`@ControllerAdvice`** bằng cách sử dụng **`@Order`** annotation hoặc implement **`Ordered`** interface.
  - \* **Lưu ý:** Nếu một ngoại lệ được xử lý bởi **`@ExceptionHandler`** cục bộ trong Controller, nó sẽ không được xử lý bởi **`@ControllerAdvice`**.
3. **`HandlerExceptionResolver` Implementations:**
  - \* Spring cung cấp một số triển khai mặc định của **`HandlerExceptionResolver`** để xử lý các loại ngoại lệ phổ biến:
    - \* **`ExceptionHandlerExceptionHandlerResolver`**: Xử lý các ngoại lệ được ném ra từ các phương thức **`@ExceptionHandler`**.
    - \* **`ResponseStatusExceptionHandlerResolver`**: Xử lý các ngoại lệ được đánh dấu **`@ResponseStatus`** hoặc **`ResponseStatusException`**.
    - \* **`DefaultHandlerExceptionHandlerResolver`**: Xử lý các ngoại lệ chuẩn của Spring MVC (ví dụ: **`HttpRequestMethodNotSupportedException`**, **`MissingServletRequestParameterException`**) và ánh xạ chúng thành các mã trạng thái HTTP phù hợp.
  - \* Các resolver này được đăng ký tự động bởi Spring Boot và có một thứ tự ưu tiên nội bộ.
4. **`ErrorController`** (hoặc **`/error`** endpoint):
  - \* Đây là cơ chế cuối cùng để xử lý lỗi. Khi một ngoại lệ không được xử lý bởi bất kỳ **`ExceptionHandler`** hoặc **`HandlerExceptionResolver`** nào, hoặc khi một lỗi HTTP (ví dụ: 404, 500) xảy ra ở tầng Servlet Container, request sẽ được chuyển tiếp đến **`/error`** endpoint.
  - \* **`BasicErrorController`** mặc định của Spring Boot sẽ xử lý endpoint này và trả về một trang lỗi HTML hoặc JSON mặc định.
  - \* Bạn có thể tùy chỉnh **`ErrorController`** hoặc tạo các trang lỗi tĩnh (ví dụ: **`404.html`**, **`500.html`**) trong **`src/main/resources/static/error/`**.

**Ví dụ về thứ tự ưu tiên:**

Nếu bạn có **`ProductNotFoundException`**:

- \* Nếu **`ProductController`** có **`@ExceptionHandler(ProductNotFoundException.class)`**, nó sẽ được sử dụng.

- \* Nếu không, và `ExceptionHandler`` (được đánh dấu `@RestControllerAdvice``) có `ExceptionHandler(ProductNotFoundException.class)``, nó sẽ được sử dụng.
- \* Nếu không có handler cụ thể nào, và `ProductNotFoundException`` được đánh dấu `@ResponseStatus(HttpStatus.NOT_FOUND)``, `ResponseStatusExceptionHandlerResolver`` sẽ xử lý nó.
- \* Nếu không có gì cả, lỗi sẽ được chuyển đến `ExceptionHandler``.

Việc hiểu rõ thứ tự này giúp bạn thiết kế một chiến lược xử lý ngoại lệ hiệu quả và tránh các hành vi không mong muốn.

## ## 5.7. Best Practices (Các phương pháp hay nhất) cho Exception Handling

Để xây dựng một hệ thống xử lý ngoại lệ mạnh mẽ, dễ bảo trì và thân thiện với người dùng, hãy tuân thủ các phương pháp hay nhất sau:

- Sử dụng `@ControllerAdvice`` / `@RestControllerAdvice`` để xử lý toàn cục:\*\***
  - \* Đây là cách tốt nhất để tập trung logic xử lý lỗi, giúp **Controller** của bạn sạch sẽ và chỉ tập trung vào logic nghiệp vụ.
  - \* Tránh lặp lại mã xử lý lỗi trong từng Controller.
- Tạo Custom Exceptions cho lỗi nghiệp vụ:\*\***
  - \* Đừng sử dụng các `RuntimeException`` chung chung như `IllegalArgumentException`` cho các lỗi nghiệp vụ cụ thể.
  - \* Tạo các `Custom Exception`` rõ ràng, có ý nghĩa (ví dụ: `UserNotFoundException``, `InsufficientStockException``). Điều này giúp mã nguồn dễ đọc, dễ hiểu và dễ gỡ lỗi hơn.
- Ánh xạ Custom Exceptions tới HTTP Status Codes phù hợp:\*\***
  - \* Sử dụng `@ResponseStatus`` trên `Custom Exception`` hoặc `ExceptionHandler`` trong `@ControllerAdvice`` để trả về mã trạng thái HTTP chính xác (ví dụ: 400 Bad Request, 404 Not Found, 409 Conflict, 403 Forbidden, 500 Internal Server Error).
  - \* Điều này giúp client hiểu rõ loại lỗi và xử lý tương ứng.
- Trả về cấu trúc phản hồi lỗi chuẩn (Standard Error Response Structure):\*\***
  - \* Luôn trả về một định dạng lỗi nhất quán (ví dụ: JSON) cho tất cả các lỗi API. Cấu trúc này nên bao gồm các thông tin như `timestamp``, `status``, `error message``, `path``, và có thể là `error code`` hoặc `details`` (cho lỗi validation).
  - \* Điều này giúp client dễ dàng phân tích và hiển thị lỗi.
- Phân biệt lỗi Client (4xx) và lỗi Server (5xx):\*\***
  - \* **Lỗi 4xx (Client Errors):\*\*** Xảy ra khi client gửi request không hợp lệ (ví dụ: dữ liệu thiếu, định dạng sai, tài nguyên không tồn tại). Client có thể sửa lỗi và gửi lại request.

- \* **Lỗi 5xx (Server Errors):** Xảy ra khi có vấn đề ở phía server (ví dụ: lỗi logic, lỗi cơ sở dữ liệu, lỗi cấu hình). Client không thể sửa lỗi này.

- \* Việc phân biệt rõ ràng giúp client và hệ thống giám sát hoạt động hiệu quả hơn.

#### 6. **Sử dụng `Problem Details` (RFC 7807) cho REST APIs:**

- \* Đối với các RESTful API hiện đại, hãy cân nhắc sử dụng định dạng `Problem Details` (RFC 7807) để trả về thông tin lỗi. Spring Boot hỗ trợ điều này thông qua thư viện `spring-web` và `spring-boot-starter-web`.

- \* `Problem Details` cung cấp một cách chuẩn hóa để mang theo thông tin chi tiết về lỗi HTTP trong phản hồi.

#### 7. **Logging ngoại lệ đúng cách:**

- \* Luôn log các ngoại lệ xảy ra ở phía server. Sử dụng các cấp độ log phù hợp (ví dụ: `ERROR` cho các lỗi nghiêm trọng, `WARN` cho các vấn đề có thể phục hồi).

- \* Đảm bảo log chứa đủ thông tin để gỡ lỗi (stack trace, thông tin request, user ID nếu có).

- \* **Không** trả về stack trace đầy đủ cho client trong môi trường production vì lý do bảo mật.

#### 8. **Tránh bắt `Exception` chung chung:**

- \* Tránh sử dụng `catch (Exception e)` hoặc `catch (Throwable e)` nếu bạn không có kế hoạch xử lý cụ thể cho tất cả các loại ngoại lệ. Điều này có thể che giấu các lỗi nghiêm trọng và làm cho việc gỡ lỗi khó khăn hơn.

- \* Hãy bắt các ngoại lệ cụ thể mà bạn biết cách xử lý.

#### 9. **Sử dụng `try-catch` cho các lỗi có thể phục hồi:**

- \* Nếu một lỗi có thể được phục hồi (ví dụ: lỗi kết nối mạng tạm thời, dữ liệu không tìm thấy nhưng có thể tạo mới), hãy sử dụng `try-catch` để xử lý và phục hồi một cách duyên dáng.

- \* Nếu lỗi không thể phục hồi, hãy để nó được ném ra và xử lý ở tầng cao hơn (ví dụ: bởi `@ControllerAdvice`).

#### 10. **Kiểm tra và xử lý `ConstraintViolationException` cho Validation:**

- \* Khi sử dụng Bean Validation, các lỗi xác thực sẽ được thu thập trong `BindingResult` hoặc ném ra `MethodArgumentNotValidException` (cho `@RequestBody`) hoặc `ConstraintViolationException` (cho `@RequestParam`, `@PathVariable`).

- \* Đảm bảo bạn có handler phù hợp để chuyển đổi các lỗi này thành phản hồi thân thiện với người dùng.

Bằng cách áp dụng các phương pháp này, bạn có thể xây dựng một hệ thống xử lý ngoại lệ mạnh mẽ, giúp ứng dụng của bạn ổn định, an toàn và dễ sử dụng hơn.